

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



MÔN HỌC: ĐỒ HỌA MÁY TÍNH

**Kỹ thuật chiếu sáng, ánh xạ texture và phản xạ
môi trường trong Three.js**

Giảng viên hướng dẫn: TS. CÁP PHẠM ĐÌNH THẮNG

Sinh viên thực hiện: Nguyễn Thế Luân

Mã sinh viên: 23520899

Lớp: CS105.Q22

Thành phố Hồ Chí Minh, tháng 04 năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp.HCM, ngày tháng 12 năm 2025

GVHD

TS. Cáp Phạm Đình Thăng

Mục lục

1	Giới thiệu	3
2	Mục tiêu	3
3	Cơ sở lý thuyết	3
3.1	Three.js và WebGL	3
3.2	Scene, Camera và Renderer	4
3.3	Geometry, Material và Mesh	4
3.4	MeshStandardMaterial	4
3.5	Texture Mapping	5
3.6	Roughness Map	5
3.7	Cubemap và Environment Mapping	5
3.8	SpotLight	6
3.9	dat.GUI	6
3.10	OrbitControls	6
4	Nội dung thực hiện	6
4.1	Tổng quan chương trình	6
4.2	Khởi tạo scene và GUI	6
4.3	Tạo vật liệu	7
4.4	Tạo hình cầu	7
4.5	Tạo mặt phẳng	7
4.6	Thiết lập ánh sáng	7
4.7	Tải cubemap	8
4.8	Áp dụng texture cho mặt phẳng	8
4.9	Áp dụng roughness map cho hình cầu	9
4.10	Điều khiển bằng dat.GUI	9
4.11	Thiết lập camera	9
4.12	Khởi tạo renderer	10
4.13	Sử dụng OrbitControls	10
4.14	Vòng lặp render	10
5	Phân tích kết quả	10
5.1	Kết quả hiển thị scene	10
5.2	Kết quả áp dụng texture cho mặt phẳng	11
5.3	Kết quả sử dụng roughness map	11
5.4	Kết quả sử dụng cubemap	12
5.5	Kết quả điều chỉnh ánh sáng	12
5.6	Kết quả điều chỉnh roughness và metalness	13
5.7	Nhận xét tổng quát	13
6	Demo và mã nguồn	13

Danh sách hình vẽ

1	Scene ban đầu với hình cầu, mặt phẳng và cubemap	11
2	Mặt phẳng sau khi áp dụng texture bê tông	11
3	Hình cầu sử dụng roughness map	12
4	Hiệu ứng phản chiếu môi trường trên hình cầu	12

1 Giới thiệu

Trong đồ họa 3D, ánh sáng và texture là hai yếu tố quan trọng giúp đối tượng hiển thị chân thực hơn. Nếu chỉ sử dụng hình học cơ bản và màu đơn sắc, scene thường có cảm giác phẳng và thiếu chiều sâu. Khi kết hợp vật liệu, ánh sáng, texture và môi trường phản chiếu, các đối tượng trong không gian 3D trở nên sinh động và gần với thực tế hơn.

Trong bài Lab 06, chương trình được xây dựng bằng thư viện Three.js nhằm minh họa kỹ thuật chiếu sáng và ánh xạ texture trong môi trường WebGL. Scene bao gồm một hình cầu đặt trên mặt phẳng, hai nguồn sáng dạng SpotLight, texture bê tông cho mặt phẳng, texture fingerprint dùng làm roughness map cho hình cầu, cùng cubemap dùng làm background và môi trường phản chiếu.

Thông qua bài thực hành, sinh viên có thể hiểu rõ hơn cách ánh sáng tương tác với vật liệu, cách texture ảnh hưởng đến bề mặt đối tượng, cũng như vai trò của các tham số roughness và metalness trong mô hình vật liệu PBR.

2 Mục tiêu

Bài Lab 06 được thực hiện nhằm đạt được các mục tiêu sau:

- Tìm hiểu kỹ thuật chiếu sáng trong Three.js.
- Sử dụng SpotLight để tạo nguồn sáng có hướng.
- Áp dụng texture lên vật thể 3D.
- Sử dụng texture bê tông cho mặt phẳng nền.
- Sử dụng roughness map để làm thay đổi độ nhám của hình cầu.
- Tạo môi trường phản chiếu bằng cubemap.
- Tìm hiểu các thuộc tính vật liệu như roughness, metalness và envMap.
- Sử dụng dat.GUI để điều chỉnh ánh sáng và vật liệu theo thời gian thực.
- Sử dụng OrbitControls để quan sát scene từ nhiều góc nhìn khác nhau.

3 Cơ sở lý thuyết

3.1 Three.js và WebGL

Three.js là thư viện JavaScript hỗ trợ xây dựng đồ họa 3D trên nền WebGL. Thư viện này giúp đơn giản hóa quá trình tạo scene, camera, renderer, ánh sáng, vật liệu và đối tượng 3D.

Trong chương trình, Three.js được sử dụng để tạo một scene 3D gồm hình cầu, mặt phẳng, ánh sáng, texture và cubemap.

3.2 Scene, Camera và Renderer

Một chương trình Three.js cơ bản thường bao gồm ba thành phần chính:

- **Scene**: không gian chứa toàn bộ đối tượng, ánh sáng và môi trường.
- **Camera**: xác định góc nhìn của người quan sát.
- **Renderer**: chịu trách nhiệm render scene lên trình duyệt.

Camera được sử dụng trong bài là `PerspectiveCamera`, giúp mô phỏng góc nhìn phối cảnh giống mắt người.

3.3 Geometry, Material và Mesh

Trong Three.js, một đối tượng 3D thường được tạo bằng cách kết hợp Geometry và Material để tạo thành Mesh.

- **Geometry**: xác định hình dạng đối tượng.
- **Material**: xác định đặc tính bề mặt.
- **Mesh**: đối tượng hoàn chỉnh được thêm vào scene.

Trong bài lab, chương trình sử dụng:

- `SphereGeometry` để tạo hình cầu.
- `PlaneGeometry` để tạo mặt phẳng.
- `MeshStandardMaterial` để vật liệu có khả năng phản ứng với ánh sáng.

3.4 MeshStandardMaterial

`MeshStandardMaterial` là vật liệu dựa trên mô hình PBR, cho phép mô phỏng bề mặt vật thể chân thực hơn thông qua các thuộc tính như:

- **roughness**: điều chỉnh độ nhám của bề mặt.
- **metalness**: điều chỉnh mức độ kim loại của vật liệu.
- **map**: texture màu chính của vật liệu.
- **bumpMap**: texture tạo hiệu ứng gồ ghề bề mặt.
- **roughnessMap**: texture điều khiển độ nhám theo từng vùng.
- **envMap**: texture môi trường dùng cho phản chiếu.

3.5 Texture Mapping

Texture Mapping là kỹ thuật phủ ảnh lên bề mặt vật thể 3D. Kỹ thuật này giúp tăng độ chi tiết cho vật thể mà không cần tăng số lượng polygon.

Trong bài lab, texture bê tông được sử dụng cho mặt phẳng nền:

```
planeMaterial.map = concreteTexture;  
planeMaterial.bumpMap = concreteTexture;  
planeMaterial.roughnessMap = concreteTexture;
```

Texture này giúp mặt phẳng có cảm giác giống bề mặt bê tông thật hơn.

3.6 Roughness Map

Roughness map là texture dùng để điều khiển độ nhám của vật liệu theo từng vùng. Những vùng sáng trong roughness map thường biểu thị bề mặt nhám hơn, còn vùng tối biểu thị bề mặt bóng hơn.

Trong chương trình, texture fingerprint được dùng làm roughness map cho hình cầu:

```
sphereMaterial.roughnessMap = fingerprintTexture;
```

Điều này tạo cảm giác bề mặt hình cầu có các vết dầu, vết tay hoặc vùng phản xạ không đồng đều, giúp vật thể trông thực tế hơn.

3.7 Cubemap và Environment Mapping

Cubemap là tập hợp sáu ảnh tương ứng với sáu hướng trong không gian 3D. Cubemap thường được dùng làm background hoặc môi trường phản chiếu.

Trong bài lab, cubemap được tải bằng `CubeTextureLoader`:

```
var urls = [  
    path + 'dark-s_px.jpg',  
    path + 'dark-s_nx.jpg',  
    path + 'dark-s_py.jpg',  
    path + 'dark-s_ny.jpg',  
    path + 'dark-s_pz.jpg',  
    path + 'dark-s_nz.jpg'  
];  
  
var reflectionCube = new THREE.CubeTextureLoader().load(urls);
```

Sau khi tải, cubemap được gán làm background của scene và môi trường phản chiếu cho vật liệu:

```
scene.background = texture;  
sphereMaterial.envMap = texture;  
planeMaterial.envMap = texture;
```

3.8 SpotLight

SpotLight là nguồn sáng dạng chùm, tương tự như đèn sân khấu hoặc đèn pin. Ánh sáng phát ra từ một vị trí và chiếu theo một hướng nhất định.

Trong bài lab, hai nguồn SpotLight được sử dụng:

```
var lightLeft = getSpotLight(2, 'rgb(255, 220, 180)');  
var lightRight = getSpotLight(2, 'rgb(255, 220, 180)');
```

Hai nguồn sáng được đặt ở hai phía khác nhau để tạo hiệu ứng chiếu sáng đều và làm nổi bật hình cầu.

3.9 dat.GUI

dat.GUI là thư viện hỗ trợ tạo giao diện điều khiển nhanh cho các tham số trong chương trình. Trong bài lab, dat.GUI được dùng để điều chỉnh:

- Cường độ ánh sáng.
- Vị trí nguồn sáng.
- Roughness của vật liệu.
- Metalness của vật liệu.

3.10 OrbitControls

OrbitControls cho phép người dùng xoay, phóng to, thu nhỏ và di chuyển góc nhìn bằng chuột. Nhờ đó, người dùng có thể quan sát ảnh hưởng của ánh sáng và texture từ nhiều hướng khác nhau.

4 Nội dung thực hiện

4.1 Tổng quan chương trình

Chương trình xây dựng một scene 3D gồm hình cầu đặt trên mặt phẳng. Hình cầu sử dụng vật liệu `MeshStandardMaterial` với roughness map và environment map. Mặt phẳng sử dụng texture bê tông, bump map và roughness map để tạo bề mặt có độ chi tiết cao hơn.

Scene còn được bổ sung cubemap làm nền và làm môi trường phản chiếu. Hai nguồn sáng SpotLight được đặt hai bên để chiếu sáng vật thể. Người dùng có thể điều chỉnh các tham số ánh sáng và vật liệu thông qua dat.GUI.

4.2 Khởi tạo scene và GUI

Scene được tạo bằng:

```
var scene = new THREE.Scene();  
var gui = new dat.GUI();
```

Scene là nơi chứa toàn bộ đối tượng 3D, ánh sáng và background. dat.GUI được sử dụng để tạo bảng điều khiển trực quan.

4.3 Tạo vật liệu

Chương trình tạo vật liệu cho hình cầu và mặt phẳng bằng hàm `getMaterial()`:

```
var sphereMaterial = getMaterial('standard', 'rgb(255, 255, 255)');  
var planeMaterial = getMaterial('standard', 'rgb(255, 255, 255)');
```

Kiểu vật liệu được sử dụng là `MeshStandardMaterial`, phù hợp cho các hiệu ứng ánh sáng, phản chiếu và texture.

4.4 Tạo hình cầu

Hình cầu được tạo bằng `SphereGeometry`:

```
var sphere = getSphere(sphereMaterial, 1, 32);  
sphere.position.y = 1;  
scene.add(sphere);
```

Hình cầu có bán kính bằng 1 và được đặt tại vị trí $y = 1$ để nằm phía trên mặt phẳng.

4.5 Tạo mặt phẳng

Mặt phẳng được tạo với kích thước 30:

```
var plane = getPlane(planeMaterial, 30);  
plane.rotation.x = -Math.PI / 2;  
scene.add(plane);
```

Do `PlaneGeometry` mặc định nằm theo phương đứng, chương trình xoay mặt phẳng quanh trục X một góc $-\pi/2$ để mặt phẳng nằm ngang.

4.6 Thiết lập ánh sáng

Chương trình sử dụng hai nguồn `SpotLight` và một nguồn `AmbientLight`:

```
var lightLeft = getSpotLight(2, 'rgb(255, 220, 180)');  
var lightRight = getSpotLight(2, 'rgb(255, 220, 180)');  
var ambientLight = new THREE.AmbientLight(0xffffff, 0.35);
```

Hai nguồn `SpotLight` được đặt ở hai phía:

```
lightLeft.position.set(-5, 5, -4);  
lightRight.position.set(5, 5, -4);
```

`AmbientLight` được dùng để bổ sung ánh sáng nền, tránh việc các vùng khuất quá tối.

4.7 Tải cubemap

Cubemap được tải từ thư mục `assets/cubemap`:

```
var path = './assets/cubemap/';
var format = '.jpg';

var urls = [
    path + 'dark-s_px' + format,
    path + 'dark-s_nx' + format,
    path + 'dark-s_py' + format,
    path + 'dark-s_ny' + format,
    path + 'dark-s_pz' + format,
    path + 'dark-s_nz' + format
];
```

Sau khi load thành công, cubemap được gán vào `background` và `envMap`:

```
scene.background = texture;

sphereMaterial.envMap = texture;
planeMaterial.envMap = texture;

sphereMaterial.needsUpdate = true;
planeMaterial.needsUpdate = true;
```

Nếu cubemap không load được, chương trình sử dụng màu xanh làm background dự phòng.

4.8 Áp dụng texture cho mặt phẳng

Texture bê tông được tải bằng `TextureLoader`:

```
var concreteTexture = loader.load('./assets/textures/concrete.jpg');
```

Texture được lặp lại nhiều lần trên mặt phẳng:

```
concreteTexture.wrapS = THREE.RepeatWrapping;
concreteTexture.wrapT = THREE.RepeatWrapping;
concreteTexture.repeat.set(15, 15);
```

Sau đó texture được áp dụng cho material:

```
planeMaterial.map = concreteTexture;
planeMaterial.bumpMap = concreteTexture;
planeMaterial.roughnessMap = concreteTexture;
planeMaterial.bumpScale = 0.01;
planeMaterial.metalness = 0.1;
planeMaterial.roughness = 0.7;
```

Nhờ đó, mặt phẳng có màu sắc, độ nhám và hiệu ứng bề mặt giống vật liệu bê tông hơn.

4.9 Áp dụng roughness map cho hình cầu

Texture fingerprint được tải và gán làm roughness map:

```
var fingerprintTexture = loader.load('./assets/textures/fingerprints.jpg');

sphereMaterial.roughnessMap = fingerprintTexture;
sphereMaterial.metalness = 0.8;
sphereMaterial.roughness = 0.15;
```

Điều này giúp hình cầu có độ phản xạ không đồng đều, tạo cảm giác bề mặt có vết bẩn hoặc dầu vân tay.

4.10 Điều khiển bằng dat.GUI

dat.GUI được dùng để tạo các nhóm điều khiển cho ánh sáng và vật liệu:

```
var folder1 = gui.addFolder('Left Light');
folder1.add(lightLeft, 'intensity', 0, 10);
folder1.add(lightLeft.position, 'x', -15, 15);
folder1.add(lightLeft.position, 'y', 0, 15);
folder1.add(lightLeft.position, 'z', -15, 15);
```

Người dùng có thể thay đổi cường độ và vị trí của từng nguồn sáng. Ngoài ra, chương trình cũng cho phép điều chỉnh roughness và metalness:

```
var folder3 = gui.addFolder('Materials');
folder3.add(sphereMaterial, 'roughness', 0, 1);
folder3.add(sphereMaterial, 'metalness', 0, 1);
folder3.add(planeMaterial, 'roughness', 0, 1);
folder3.add(planeMaterial, 'metalness', 0, 1);
```

4.11 Thiết lập camera

Camera được tạo bằng PerspectiveCamera:

```
var camera = new THREE.PerspectiveCamera(
    45,
    window.innerWidth / window.innerHeight,
    0.1,
    1000
);

camera.position.set(0, 2, 8);
camera.lookAt(0, 1, 0);
```

Camera được đặt phía trước scene và nhìn vào hình cầu.

4.12 Khởi tạo renderer

Renderer được tạo bằng WebGLRenderer:

```
var renderer = new THREE.WebGLRenderer({
    antialias: true
});

renderer.outputColorSpace = THREE.SRGBColorSpace;
renderer.setClearColor(0x87ceeb);
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
```

Tùy chọn `antialias` giúp làm mượt cạnh vật thể. `outputColorSpace` giúp màu sắc hiển thị đúng hơn khi sử dụng texture.

4.13 Sử dụng OrbitControls

OrbitControls được thêm vào để người dùng có thể xoay và quan sát scene bằng chuột:

```
var controls = new THREE.OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
```

Nhờ đó, việc quan sát hiệu ứng ánh sáng, phản chiếu và texture trở nên trực quan hơn.

4.14 Vòng lặp render

Scene được render liên tục bằng hàm `update()`:

```
function update(renderer, scene, camera, controls) {
    controls.update();
    renderer.render(scene, camera);

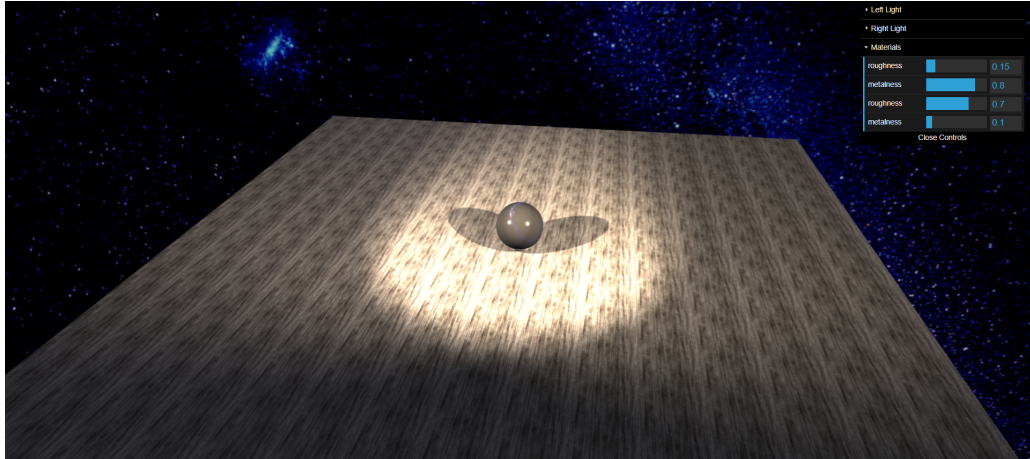
    requestAnimationFrame(function() {
        update(renderer, scene, camera, controls);
    });
}
```

Vòng lặp này giúp scene luôn được cập nhật khi người dùng thay đổi góc nhìn hoặc điều chỉnh thông số trong `dat.GUI`.

5 Phân tích kết quả

5.1 Kết quả hiển thị scene

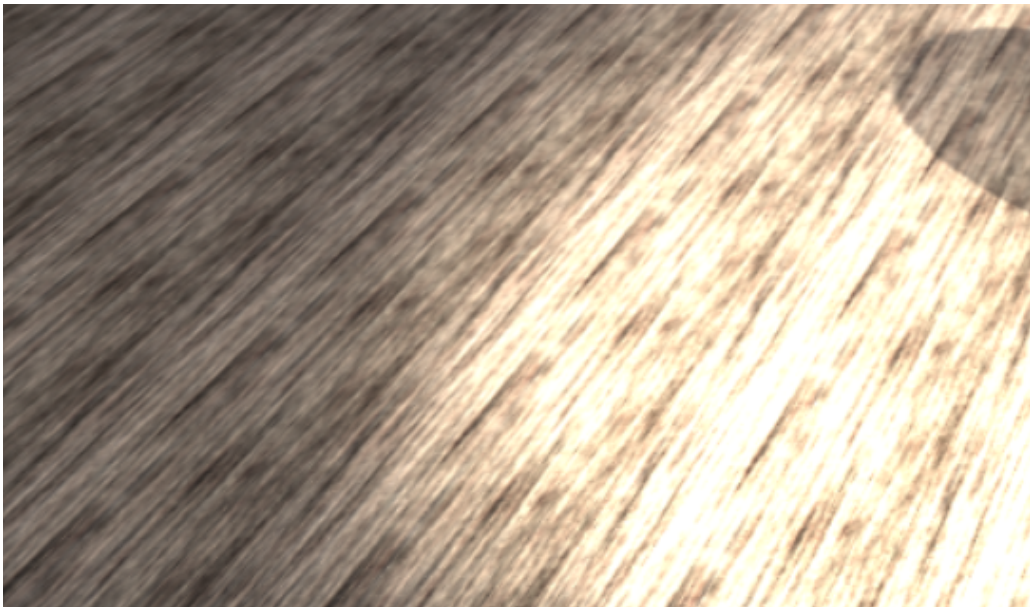
Sau khi chạy chương trình, scene hiển thị một hình cầu nằm trên mặt phẳng. Background được tạo bằng cubemap, giúp không gian 3D có cảm giác bao quanh thay vì chỉ là một màu nền đơn giản.



Hình 1: Scene ban đầu với hình cầu, mặt phẳng và cubemap

5.2 Kết quả áp dụng texture cho mặt phẳng

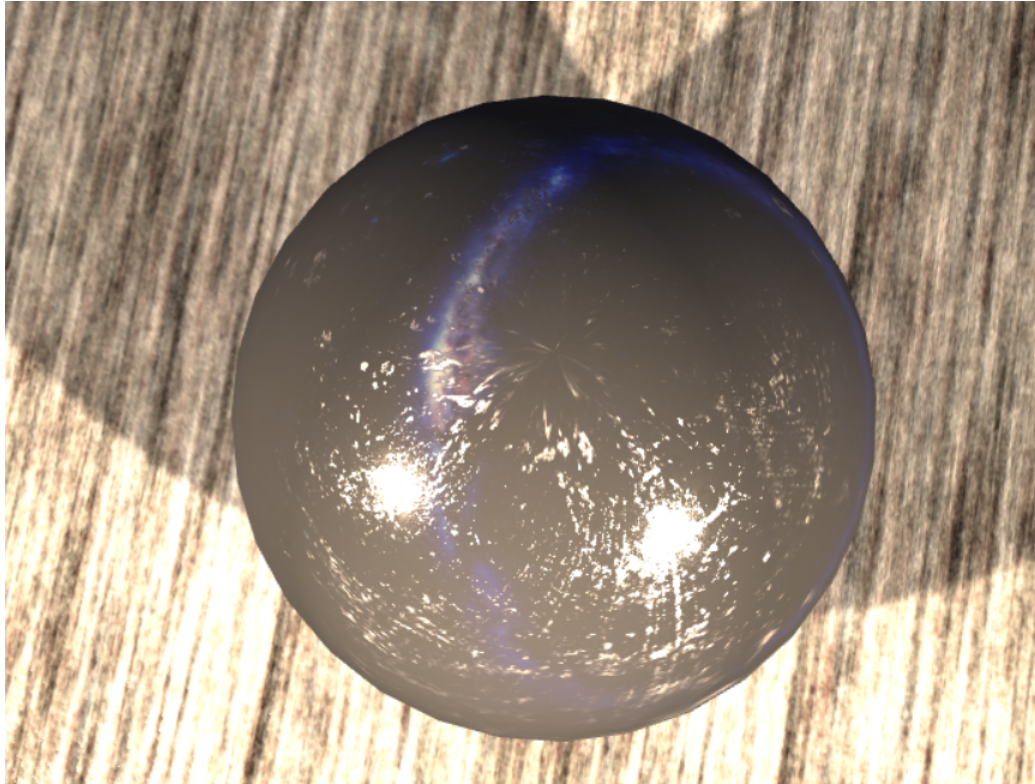
Mặt phẳng sử dụng texture bê tông nên bề mặt có nhiều chi tiết hơn so với material màu đơn sắc. Việc sử dụng `RepeatWrapping` giúp texture được lặp lại trên toàn bộ mặt phẳng, tránh tình trạng ảnh bị kéo giãn quá lớn.



Hình 2: Mặt phẳng sau khi áp dụng texture bê tông

5.3 Kết quả sử dụng roughness map

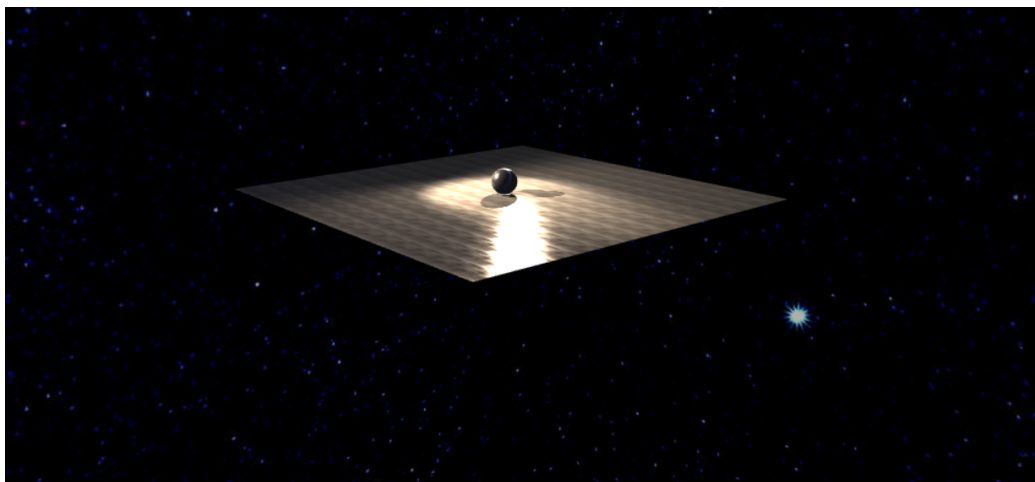
Khi áp dụng texture fingerprint làm roughness map, hình cầu có độ phản xạ thay đổi theo từng vùng. Một số vùng phản xạ mạnh hơn, trong khi những vùng khác mờ hơn. Điều này giúp vật thể trông tự nhiên và bớt giả hơn so với bề mặt bóng đều toàn bộ.



Hình 3: Hình cầu sử dụng roughness map

5.4 Kết quả sử dụng cubemap

Cubemap đóng vai trò làm môi trường xung quanh scene và tạo hiệu ứng phản chiếu trên vật liệu. Khi hình cầu có **metalness** cao và **roughness** thấp, hiệu ứng phản chiếu từ cubemap trở nên rõ ràng hơn.



Hình 4: Hiệu ứng phản chiếu môi trường trên hình cầu

5.5 Kết quả điều chỉnh ánh sáng

Khi thay đổi cường độ hoặc vị trí của hai nguồn SpotLight trong dat.GUI, vùng sáng và vùng tối trên hình cầu thay đổi rõ rệt. Điều này cho thấy ánh sáng có ảnh hưởng trực

tiếp đến cách vật liệu được hiển thị.

- Khi tăng intensity, hình cầu và mặt phẳng sáng hơn.
- Khi thay đổi vị trí đèn, vùng highlight trên hình cầu thay đổi.
- Hai nguồn sáng hai bên giúp scene có chiều sâu và cân bằng hơn.

5.6 Kết quả điều chỉnh roughness và metalness

Khi thay đổi roughness và metalness bằng dat.GUI, có thể quan sát rõ sự khác biệt:

- Roughness thấp: vật thể bóng hơn, phản chiếu rõ hơn.
- Roughness cao: vật thể mờ hơn, phản chiếu kém hơn.
- Metalness thấp: vật thể giống nhựa hoặc vật liệu phi kim.
- Metalness cao: vật thể giống kim loại hơn và phản chiếu môi trường mạnh hơn.

5.7 Nhận xét tổng quát

Kết quả cho thấy chương trình đã thể hiện đúng mục tiêu của bài lab về chiếu sáng và texture. Việc kết hợp SpotLight, texture mapping, roughness map và cubemap giúp scene có chiều sâu và tính chân thực cao hơn.

Thông qua dat.GUI, người dùng có thể điều chỉnh trực tiếp các tham số ánh sáng và vật liệu để quan sát sự thay đổi theo thời gian thực. Điều này giúp hiểu rõ hơn mối quan hệ giữa ánh sáng, vật liệu và môi trường trong đồ họa 3D.

6 Demo và mã nguồn

Để minh họa cho kết quả của bài lab, chương trình đã được triển khai và chia sẻ thông qua các đường dẫn sau:

- **Link web demo:** <https://cs-105-q22-chfh.vercel.app/>
- **Link GitHub:** CS105.Q22 - Lab-06
- **Video demo:** Video Demo